Deep learning model evaluation & visualisation



Amina Mardiyyah Rufai

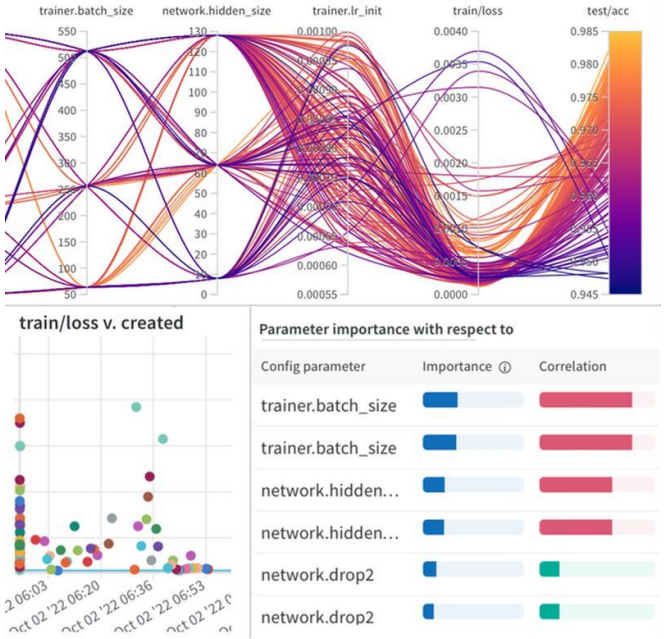
Machine learning Data Scientist

EMBL-EBI

From training models to understanding model behaviour

What we will cover

- ➔ Why model evaluation matters in deep learning workflows
- ➔ Training, validation, and test performance
- ➔ Common metrics for classification and NLP tasks
- ➔ Visualising model behaviour and failure modes
- ➔ Comparing models using quantitative and qualitative analysis
- ➔ Tracking experiments using Weights & Biases
- ➔ Hands-on: evaluate and visualise a biomedical NLP model



“I trained a model — now how do I know whether it is good, trustworthy, and useful?”

The importance of model evaluation

Training performance is not scientific evidence or good generalisation.

A model can fit training examples very well and still fail on new data.



Model evaluation helps us diagnose:

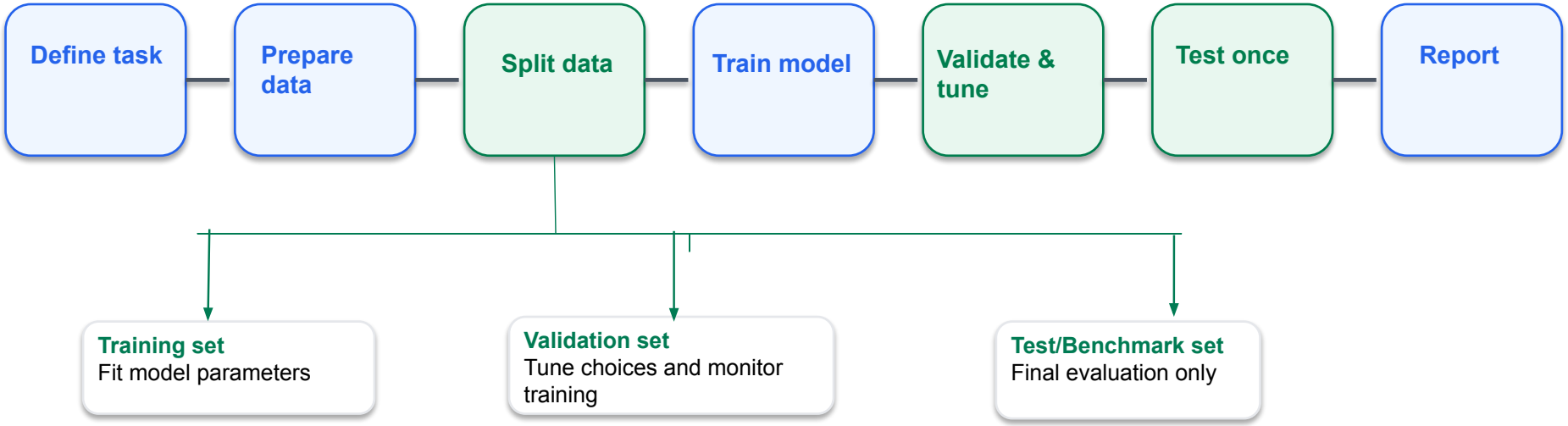
- ✓ Overfitting
- ✓ Underfitting/ poor generalisation
- ✓ class imbalance
- ✓ domain shift, etc

It also helps us detect:

- ✓ if we're using the right metrics for the right task

Train, evaluate, iterate

The complete model development lifecycle



Each of these split have different jobs

Rule of thumb: train many times, validate during development, test once at the end.

Training, validation and test sets

Different splits have different jobs

Split	Used for	What it tells us
Training	Fit model parameters	Can the model learn from examples?
Validation	Tune hyperparameters, choose checkpoints, monitor overfitting	Does performance improve on unseen development data?
Test	Final evaluation only	How well does the final model generalise?



Example split — adjust ratios based on data size, task and study design

Common Evaluation Metrics used in NLP

Not all metrics are used equally. There is no "one size fits all" metric

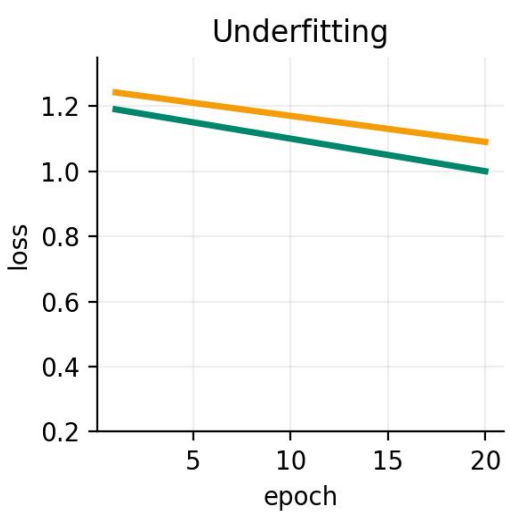
NLP task	Common metrics	Examples
Text Classification	Accuracy, F1 score, precision, recall	Sentiment analysis, disease classification, relation extraction
Token classification	Overall F1 score(micro/macro), entity-level f1 score, precision, recall	Named entity extraction, POS tagging, etc
Language modelling	Perplexity, cross-entropy loss	Text generation, next-token prediction,
Regression	MSE/RMSE Loss, correlation	Predicting review scores, clinical risk scores prediction, etc
Information retrieval	Cosine similarity	Document retrieval, semantic search, question-answer retrieval, literature search
Summarisation	ROUGE-1, ROUGE-2, ROUGE-L, BERTScore, human evaluation	Article summarisation, clinical note summarisation,
Machine translation	BLEU, Comet	Translating research papers/global health reports from one language to another

Common Model failures and how to diagnose them

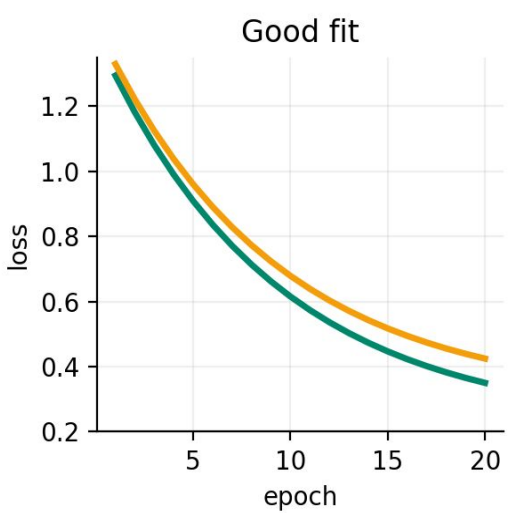
Different metrics tell different stories

Learning curves: Underfitting, Good fit, Overfitting

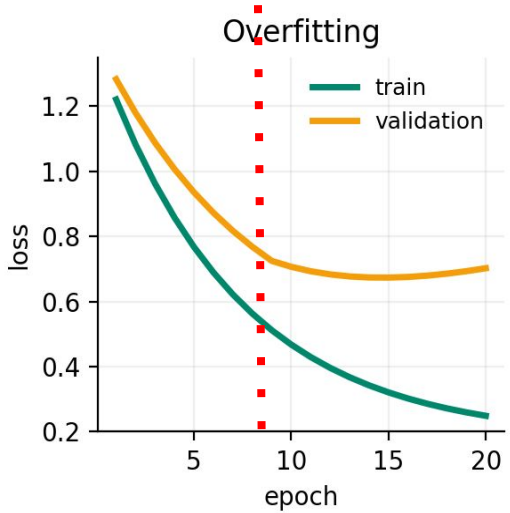
Learning curves are the first diagnostic tool for deep learning.



Both train and validation performance stay poor.



Train and validation improve together.



Training improves while validation worsens.

Learning curves show whether training is improving, generalising, or memorising.

From curve pattern to next action

A visual diagnosis should lead to a modelling decision

Diagnosis	What it means	Common fixes
Underfitting	Model is too simple or poorly trained	Check data quality, train longer, adjust/change model architecture
Overfitting	Model memorises training data	Regularise, early stop, add data, simplify model, review data splitting strategy
Good fit(The sweet spot)	Training and validation improve together	Keep model; evaluate on test once

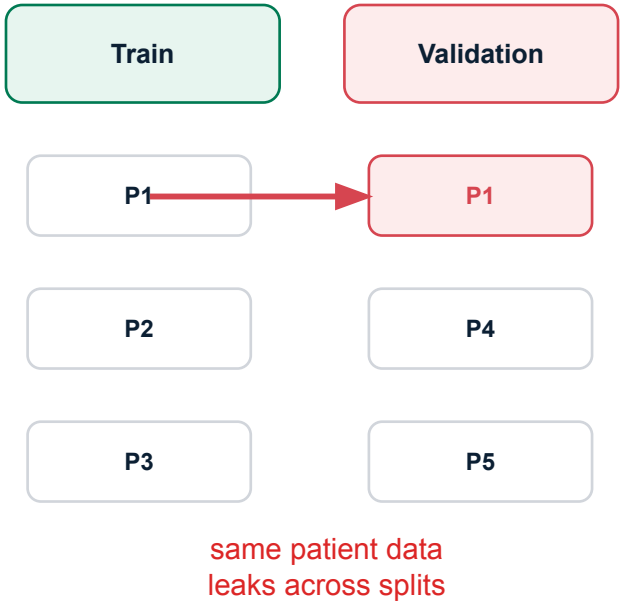
Data leakage: the silent performance booster

What it means:
Information from the validation/test set accidentally appears in training.

What it looks like:
Unrealistically good performance that doesn't generalise

Common symptoms:
- ➔ Suspiciously perfect validation/test metrics (e.g 99.9% accuracy)
- ➔ Model performs terribly in production despite great test results

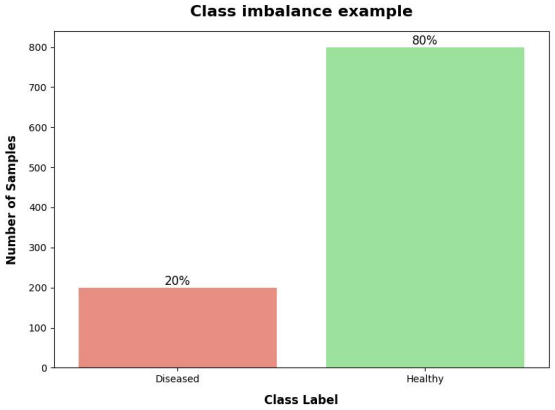
Common causes:
- ➔ Preprocessing before splitting
- ➔ Duplicate records across splits

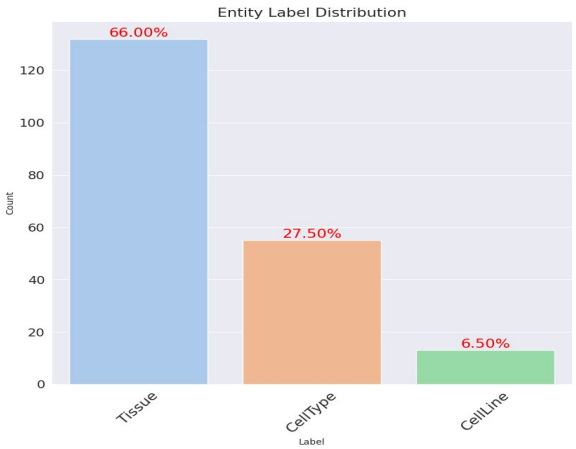


Key takeaway
Split by the correct biological, clinical or document unit — not just by row.

Class imbalance: Why it matters

Rare classes may be scientifically important but statistically under-represented.





Failure Mechanism	Description
Majority-Class Bias	The model learns to prioritise the dominant class because it contributes most to the training objective.
Misleading Accuracy	A model can achieve very high accuracy by predicting only the majority class (e.g., 90% accuracy with 0% detection of disease cases).
Poor Minority Representation	Insufficient examples limit the model's ability to learn meaningful minority-class patterns.
Biased Decision Boundaries	Classification boundaries shift toward the minority class, reducing sensitivity and recall.
Overfitting Minority Samples	When minority samples are scarce, models may memorise them rather than generalise.

High accuracy can hide failure on the class you care about.

Accuracy: where it helps — and where it hides problems

Accuracy asks: "What proportion of predictions were correct?"

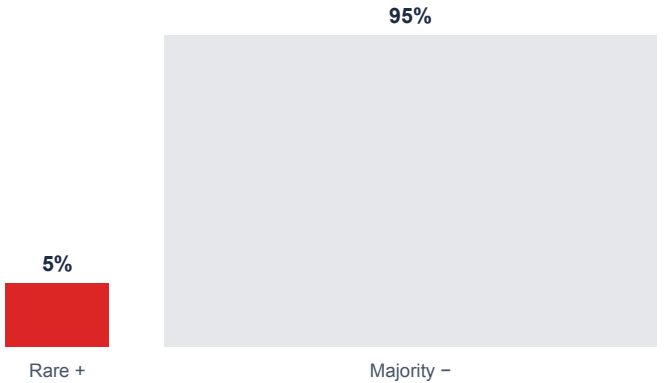
Accuracy = correct predictions / total predictions

Helpful when: balanced classes + similar error costs



Examples: balanced image classification, balanced topic classification, QC pass/fail with similar error costs.

Misleading: imbalanced data or asymmetric error costs



Not enough for rare disease detection, literature screening, NER, or tasks where false negatives and false positives have different costs.

Report accuracy when it matches the research goal; otherwise pair it with class-level metrics.

Precision, Recall, F1: Different views of error

These metrics answer different scientific questions.

Metric	Question it answers	Example
Precision	Of predicted positives, how many were correct?	Can I trust the papers/entities flagged by the model?
Recall	Of true positives, how many did we find?	How many important papers/entities did we miss?
F1-score	How well do precision and recall balance?	Useful when both false positives and false negatives matter

Precision = trust the positives

Recall = find what matters

F1 = balance both

Macro, micro and weighted F1: Averaging choice matters when classes are imbalanced

Metric	Question it answers	Where to report
Micro F1	Pools all decisions before computing F1	Overall performance across many examples
Macro F1	Computes F1 per class, then gives each class equal weight	Rare biomedical classes matter
Weighted F1	Computes F1 per class, weighted by class frequency	Performance should reflect dataset distribution

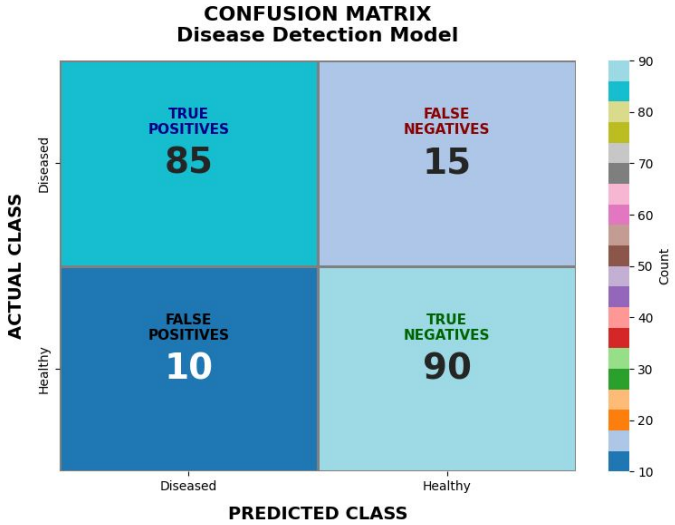
Precision = trust the positives

Recall = find what matters

F1 = balance both

Confusion Matrix: Reading model errors

A diagnostic view of true labels versus predicted labels.



Diagonal cells

Correct predictions. Larger diagonal values usually indicate better class-wise performance.

Off-diagonal cells

Errors. These tell you which classes are confused and where to inspect examples.

False negatives

Relevant evidence that the model missed. In literature screening, these may be scientifically costly.

False positives

Irrelevant examples sent for review. They cost time but may be acceptable if recall is critical.

A confusion matrix is a diagnostic tool, not a metric.

Confusion Matrix: Reading model errors

When it is useful	When it is not
Classification problem where num classes are small(e.g 2, 3, 4)	Regression problem, token classification problem num classes are many(e.g 10)
Useful to visualise actual error counts vs prediction counts	Less informative when there are many classes or hierarchical diagnoses without clear binary interpretation
Makes minority-class errors visible when accuracy is misleading.	Does not account for prevalence, so performance can still look misleading.

Error Analysis: what is the model actually doing?

Metrics tell you how much error; error analysis tells you what kind of error.

The questions to ask:

- Are certain labels harder than the other?
- Are errors concentrated in a certain label?
- Are the errors scientifically acceptable

Quantitative analysis:

- How many errors?
- Where is it happening?
- How often is it happening?

It counts the failures

Vs

Qualitative analysis- The Stories Behind Failures

- Why does it happen?
- When does it happen/ what are the triggers?
- What does failure look like?

It reveals failure patterns

These are complimentary, not competing — the best error analysis uses both

Biomedical evaluation needs extra care

A metric is only meaningful in the context of the data and research question.

Class imbalance
rare but important labels

Small datasets
high variance estimates

Annotation uncertainty
ambiguous or noisy labels

Repeated units
patients, donors, studies, batches

Domain shift
new lab, site, disease area or corpus

Ambiguous terms
context changes meaning

Evaluation implications

- ★ Deep learning models can seem confident, but;
- ★ Their probabilities are just estimates not guarantee
- ★ Sometimes, it helps to calibrate predictions

Prediction confidence

Correct + confident	**Wrong + confident**
Correct + uncertain	**Wrong + uncertain**

High-confidence errors deserve special attention in biomedical settings.

EMBL-EBI

Experiment tracking and Visualisation

Using Weights & Biases to organise runs, metrics, plots and artifacts.



Why experiment tracking?

Deep learning experiments become hard to trust when runs are not traceable.

Deep learning runs are stochastic and sensitive to small choices:

- ➔ Learning rate, batch size, epochs
- ➔ Random seed and checkpoint selection
- ➔ Dataset version and preprocessing
- ➔ Model architecture or pretrained model
- ➔ Thresholds and evaluations metrics

Without tracking

- lost runs
- unclear improvements
- hard to reproduce

Experiment tracking supports

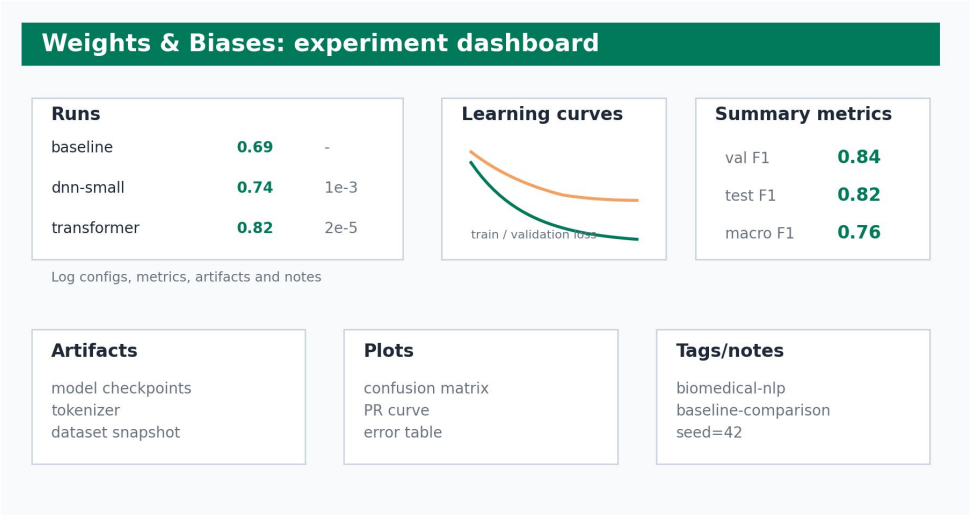
- reproducibility
- debugging
- fair comparison

Weight and Bias: From runs to evidence

W&B tracks, visualises and compares machine-learning experiments.

What you can log

- ➔ **Experiment configs**: Model name/architecture e.g Biobert
- ➔ **Training dynamics**: Learning rate, batch size, epochs, random seed training loss, training metrics etc
- ➔ **Evaluation metrics:** Accuracy, Loss plots, etc
- ➔ **Artifacts**: Dataset version, metadata etc



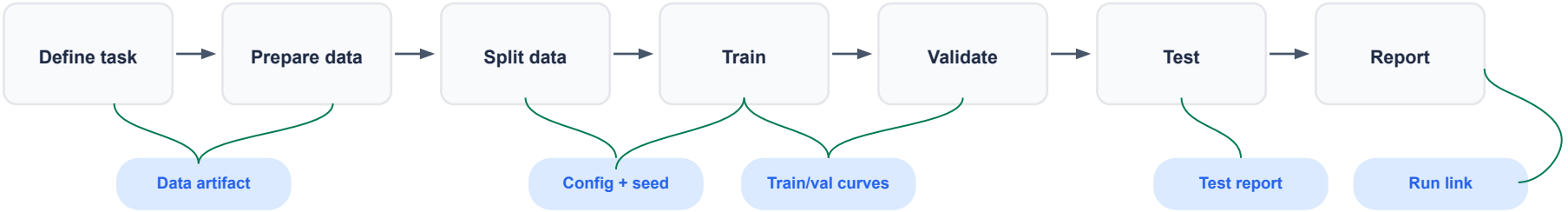
Track **Visualise** **Compare**

Log enough information to reproduce the results

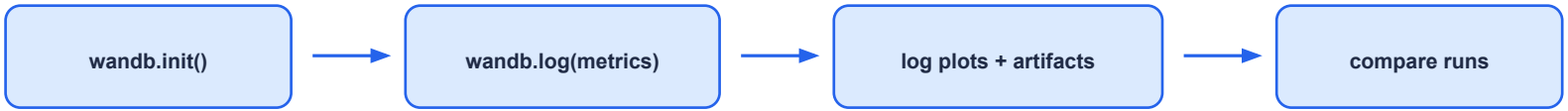
EMBL-EBI

Where W&B fits in the model lifecycle

W&B should support the whole experiment, not just the final score.



A practical W&B workflow



Hands-on Tutorial:

Evaluating and Visualising Model Predictions — Understanding Patterns, Errors, and Failure Modes

Colab Link: Here